嵌入式实时操作系统

与 I2C 传感器实验报告

——uC/OS-III 移植与 AHT20 温湿度采集

课程名称：嵌入式系统

学生姓名：刘佳烜

学生学号：632407030115

专业班级：2024 级通信工程 1 班

实验平台：STM32F103C8T6 + Keil MDK + STM32CubeMX

实验日期：2026 年 10 月

一、实验目的与任务

本实验分为两个任务。任务一学习嵌入式实时操作系统 uC/OS-III，并将其移植到 STM32F103C8T6 上，建立三个周期任务：两个 LED 任务分别以 1 s 和 3 s 周期翻转 LED，第三个任务每 2 s 通过 USART1 发送“hello uc/OS! 欢迎来到RTOS多任务环境！”。任务二学习 I2C 总线，使用 AHT20 温湿度传感器采集数据，并把任务一中的串口任务替换为 AHT20 传感器任务，每 2 s 采集一次温湿度、通过串口输出到上位机。

通过本实验，应掌握 RTOS 的启动流程、任务创建与延时、SysTick/PendSV/SVC 的移植方法，理解软件 I2C 与硬件 I2C 的区别，能够根据传感器数据手册完成初始化、触发测量、读取数据和 CRC 校验。

二、实验环境

- MCU：STM32F103C8T6（Cortex-M3，72 MHz，64 KB Flash，20 KB SRAM）
- 开发环境：STM32CubeMX、Keil MDK 5、STM32F1xx HAL 库
- RTOS：Micrium uC/OS-III（ARM Cortex-M3 Keil 端口）
- 传感器：AHT20 温湿度传感器（I2C，3.3 V）
- 其他：USB-TTL 串口模块、ST-Link、LED、4.7 kΩ 上拉电阻、杜邦线
- 上位机：Windows 10/11，串口助手（115200-8-N-1）

三、任务一：uC/OS-III 的移植

3.1 获取源码与目录规划

从 Micrium 官网下载 uC/OS-III 源码，将以下目录加入工程：

目录	作用
uC-CPU	CPU 相关定义、临界区、字节序等
uC-LIB	Micrium 公共库函数
uC-OS3/Source	os_core.c、os_task.c、os_time.c 等内核源码
uC-OS3/Ports/ARM-Cortex-M3/Generic/Keil	os_cpu_a.asm、os_cpu_c.c、os_cpu.h
Application/User/Core	main.c、app_tasks.c、soft_i2c.c、aht20.c
Drivers/STM32F1xx_HAL_Driver	HAL 库

在 Keil 中建立对应的 Group，并把所有 .c 文件和 os_cpu_a.asm 加入工程；在 Options for Target → C/C++ → Include Paths 中加入上述源码目录、HAL 目录和 Core/Inc。

3.2 STM32CubeMX 配置

新建 STM32F103C8T6 工程，完成以下配置：

- RCC：HSE 8 MHz，PLL ×9，SYSCLK=72 MHz，APB1=36 MHz，APB2=72 MHz。
- SYS：Debug 选择 Serial Wire；HAL Timebase Source 选择 TIM2，避免 HAL 的 SysTick 与 uC/OS-III 冲突。
- GPIO：PC13 推挽输出（板载 LED1），PB12 推挽输出（外接 LED2），默认输出高电平（低电平点亮）。
- USART1：PA9/PA10，115200-8-N-1；开启 USART1 全局中断可按需选择，本例使用阻塞发送。
- 生成 Keil MDK 工程，保留 main.c、stm32f1xx_it.c、HAL 初始化代码。

3.3 拷贝 RTOS 文件与修改编译选项

把 uC/OS-III 源码复制到工程 Middlewares 目录，加入 Keil 工程；在 os_cfg.h 中设置节拍和功能开关。关键配置如下：

```
#define OS_CFG_TICK_RATE_HZ          1000u
#define OS_CFG_APP_HOOKS_EN          DEF_DISABLED
#define OS_CFG_SCHED_ROUND_ROBIN_EN  DEF_DISABLED
#define OS_CFG_STAT_TASK_EN          DEF_DISABLED
#define OS_CFG_TASK_STK_REDZONE_EN   DEF_DISABLED
```

CPU_CFG.h 中按 Cortex-M3 配置 NVIC 优先级位数（STM32F1 为 4 位）、系统节拍和临界区宏；lib_cfg.h 保持默认。OS_CFG_TICK_RATE_HZ=1000 表示 1 ms 一个节拍，OSTimeDlyHMSM 的毫秒参数才有意义。

Keil 的 ARM Compiler 建议使用 V6；如果使用 AC5，需确认 os_cpu_a.asm 的汇编语法与编译器匹配。

3.4 中断向量与启动流程

CubeMX 会在 stm32f1xx_it.c 中生成空的 SysTick_Handler、PendSV_Handler 和 SVC_Handler。删除这三个空函数，改为调用 uC/OS-III 的端口函数：

```
#include "os.h"
void SysTick_Handler(void) { OS_CPU_SysTickHandler(); }
void PendSV_Handler(void)  { OS_CPU_PendSVHandler();  }
void SVC_Handler(void)     { OS_CPU_SVCHandler();     }
```

在 main() 中先完成 HAL、时钟、GPIO、USART 初始化，再调用 OSInit()、创建任务、OSStart()。OSStart() 不会返回。

```
int main(void)
{
    OS_ERR os_err;
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_USART1_UART_Init();
    OSInit(&os_err);
    AppTasks_Init();
    OSStart();
```

```
    while (1) { }
}
```

PendSV 应设为最低优先级，SysTick 的优先级按端口要求设置；STM32F1 使用 NVIC_PriorityGroup_4。若优先级设置不当，任务切换会失效或进入 HardFault。

3.5 三个任务的实现

任务栈使用 CPU_STK 数组，任务优先级数值越小越高。LED 任务每 1 s/3 s 翻转一次；串口任务每 2 s 发送一次 hello 信息。

```
#define TASK_LED1_PRIO    4u
#define TASK_LED2_PRIO    5u
#define TASK_UART_PRIO    6u
#define TASK_STK_SIZE     256u

static void Task_LED1(void *p_arg)
{
    while (1) {
        HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_13);
        OSTimeDlyHMSM(0,0,1,0,OS_OPT_TIME_HMSM_STRICT,&err);
    }
}

static void Task_LED2(void *p_arg)
{
    while (1) {
        HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_12);
        OSTimeDlyHMSM(0,0,3,0,OS_OPT_TIME_HMSM_STRICT,&err);
    }
}

static void Task_Uart(void *p_arg)
{
    char msg[] = "hello uc/OS! 欢迎来到RTOS多任务环境! \r\n";
    while (1) {
        HAL_UART_Transmit(&huart1,(uint8_t *)msg,strlen(msg),100);
        OSTimeDlyHMSM(0,0,2,0,OS_OPT_TIME_HMSM_STRICT,&err);
    }
}
```

创建任务时使用 OSTaskCreate()，并检查返回值。若返回 OS_ERR_NONE 表示创建成功；若返回 OS_ERR_TASK_CREATE_ISR，说明在中断中创建任务；若返回 OS_ERR_PRIO_INVALID，检查优先级是否超出范围。

```
OSTaskCreate(&tcb_led1, "LED1", Task_LED1, 0,
             TASK_LED1_PRIO, &stk_led1[0], TASK_STK_SIZE/10,
             TASK_STK_SIZE, 0, 0, 0,
             OS_OPT_TASK_STK_CHK | OS_OPT_TASK_STK_CLR, &err);
```

3.6 编译、下载与现象

编译工程，确认没有重复定义的 SysTick_Handler/PendSV_Handler；用 ST-Link 下载后复位。正常现象为：PC13 板载 LED 每 1 s 翻转一次，PB12 外接 LED 每 3 s 翻转一次，USART1 每 2 s 输出一行 hello 信息。

如果程序停在 HardFault，优先检查：栈是否溢出、PendSV 优先级是否最低、是否误用了 HAL_Delay、是否在中断中调用了阻塞 API、os_cpu_a.asm 是否加入工程。

四、任务二：I2C 与 AHT20 温湿度采集

4.1 软件 I2C 与硬件 I2C 的区别

对比项	软件 I2C	硬件 I2C
实现方式	GPIO 模拟 SCL/SDA 时序	STM32 的 I2C 外设自动产生时序
引脚	可任选 GPIO，布线灵活	固定复用引脚（如 PB6/PB7）
CPU 占用	较高，需程序延时和翻转	较低，可用中断/DMA
速率	通常 100 kHz 左右，受延时影响	标准/快速模式，速率稳定
调试	逻辑简单，便于观察和移植	寄存器/状态机复杂，出错难排查
可靠性	依赖时序和上拉，易受干扰	外设处理 ACK、仲裁、错误

软件 I2C 用两个开漏 GPIO 模拟时序：SCL 高电平时 SDA 由高变低为起始，SCL 高电平时 SDA 由低变高为停止；每个时钟周期传送 1 bit，第 9 个时钟接收 ACK/NACK。硬件 I2C 则由外设自动完成这些动作。

4.2 AHT20 的通信协议

AHT20 的 7 位 I2C 地址为 0x38，写地址 0x70，读地址 0x71。上电后先延时约 40 ms，读状态寄存器；若校准位未使能，发送 0xBE、0x08、0x00 进行初始化。每次测量发送 0xAC、0x33、0x00，等待约 80 ms，再读取 7 个字节：状态字节 + 5 个数据字节 + CRC 字节。

设读到的 20 位湿度原始值为 rawH，20 位温度原始值为 rawT，则：

$$RH(\%) = rawH / 2^{20} \times 100；T(℃) = rawT / 2^{20} \times 200 - 50。$$

CRC8 多项式为 0x31，初值 0xFF，对前 6 个字节计算，与第 7 个字节比较。CRC 不匹配时丢弃本次数据，避免把错误数据送到上位机。

4.3 硬件连接

AHT20 引脚	STM32F103C8T6	说明
VCC	3.3 V	不要接 5 V
GND	GND	共地
SCL	PB6	软件 I2C SCL，4.7 kΩ 上拉
SDA	PB7	软件 I2C SDA，4.7 kΩ 上拉

4.4 软件 I2C 与 AHT20 驱动

软件 I2C 的 SCL/SDA 都配置为 GPIO_MODE_OUTPUT_OD，外部接 4.7 kΩ 上拉。开漏输出写 1 时释放总线，写 0 时拉低；读取 SDA 时直接读 IDR。

```
void SoftI2C_Start(void) {
    SDA_H(); SCL_H(); DelayUs(5);
```

```
    SDA_L(); DelayUs(5); SCL_L(); DelayUs(5);
}

uint8_t SoftI2C_WriteByte(uint8_t byte) {
    for (uint8_t i=0;i<8;i++) {
        SCL_L(); if (byte & 0x80) SDA_H(); else SDA_L();
        byte <<= 1; DelayUs(2); SCL_H(); DelayUs(5);
    }
    SCL_L(); SDA_H(); DelayUs(2); SCL_H(); DelayUs(2);
    uint8_t ack = (SDA_READ()==GPIO_PIN_RESET)?0:1;
    SCL_L(); return ack;
}
```

AHT20 读取时先触发测量，再等待 80 ms；等待使用 OSTimeDlyHMSM，而不是 HAL_Delay，因为 HAL 的 SysTick 已被 uC/OS-III 接管。读取完成后做 CRC8 校验，再按公式换算温湿度。

```
uint8_t AHT20_Read(float *t, float *h) {
    uint8_t cmd[3] = {0xAC,0x33,0x00};
    uint8_t buf[7];
    AHT20_WriteBytes(cmd,3);
    OSTimeDlyHMSM(0,0,0,80,OS_OPT_TIME_HMSM_NON_STRICT,&err);
    while ((AHT20_ReadStatus() & 0x80) != 0) {
        OSTimeDlyHMSM(0,0,0,2,OS_OPT_TIME_HMSM_NON_STRICT,&err);
    }
    AHT20_ReadBytes(buf,7);
    if (AHT20_CalcCRC8(buf,6) != buf[6]) return 4;
    uint32_t rawH = ((uint32_t)buf[1]<<16 | (uint32_t)buf[2]<<8 | buf[3]) >> 4;
    uint32_t rawT = (((uint32_t)buf[3]&0x0F)<<16) | ((uint32_t)buf[4]<<8) | buf[5];
    *h = rawH * 100.0f / 1048576.0f;
    *t = rawT * 200.0f / 1048576.0f - 50.0f;
    return 0;
}
```

4.5 把串口任务改为温湿度采集

把任务一的 Task_Uart 替换为 Task_Sensor：任务启动时先初始化 AHT20，之后每 2 s 读取一次，格式化后通过 USART1 发送。为避免浮点 printf 的兼容性问题，代码把温度、湿度各放大 10 倍后用整数输出。

```
static void Task_Sensor(void *p_arg) {
    float t, h; char msg[96]; int ti, hi;
    while (AHT20_Init() != 0) OSTimeDlyHMSM(0,0,1,0,OS_OPT_TIME_HMSM_STRICT,&err);
    while (1) {
        if (AHT20_Read(&t,&h) == 0) {
            ti=(int)(t*10); hi=(int)(h*10);
            snprintf(msg,sizeof(msg),"AHT20: T=%d.%d C, RH=%d.%d %%\r\n",
                     ti/10,ti%10,hi/10,hi%10);
        } else snprintf(msg,sizeof(msg),"AHT20 read error\r\n");
        HAL_UART_Transmit(&huart1,(uint8_t *)msg,strlen(msg),100);
        OSTimeDlyHMSM(0,0,2,0,OS_OPT_TIME_HMSM_STRICT,&err);
    }
}
```

注意：AHT20_Init() 内部也使用了 OSTimeDlyHMSM，所以必须在 OSStart() 之后的任务中调用，不能放在 OSStart() 之前的 main() 里。

五、实验结果与分析

下载程序后，两个 LED 按不同周期独立闪烁，说明三个任务在 uC/OS-III 调度下并行运行；串口助手每 2 s 收到一行温湿度数据，说明任务二的阻塞延时不占 CPU，其他任务仍能按时运行。

串口输出示例（数值随环境变化）：

```
AHT20: T=26.4 C, RH=55.3 %
AHT20: T=26.4 C, RH=55.2 %
AHT20: T=26.5 C, RH=55.4 %
```

从结果可以看出：LED1 的翻转间隔约 1 s，LED2 约 3 s，串口约 2 s；三个周期互不干扰。AHT20 的温度和湿度变化平缓，CRC 校验通过，说明 I2C 时序、地址、命令和换算公式正确。

如果串口没有输出，先检查 USART1 的波特率和接线；如果 AHT20 返回错误，先检查 3.3 V 供电、PB6/PB7 上拉、地址 0x70/0x71 和初始化命令 0xBE 0x08 0x00。

六、常见问题与解决

现象	可能原因	解决方法
编译报错重复定义 SysTick_Handler	CubeMX 空函数与 port_glue.c 冲突	删除 stm32f1xx_it.c 中的空函数，只保留端口版本
程序停在 HardFault	PendSV 优先级、栈溢出、中断中调用阻塞 API	PendSV 设为最低；增大任务栈；中断中只发信号
LED 不闪	GPIO 时钟/引脚配置错误，任务未创建	检查 CubeMX GPIO；检查 OSTaskCreate 返回值
串口乱码	波特率、时钟或共地问题	USART1 设为 115200；检查 72 MHz 时钟和 GND
AHT20 无 ACK	上拉缺失、地址错误、供电不足	SDA/SCL 接 4.7 kΩ 上拉；确认 3.3 V 和 0x70 地址
温湿度为 0 或跳变	CRC 失败、命令错误、未等待 80 ms	检查 0xAC 0x33 0x00；等待 80 ms；检查 CRC8

七、实验心得

这次实验让我第一次把 RTOS 的“任务”与裸机程序里的“死循环”区分开。裸机程序通常是一个大循环加延时，CPU 在延时里空转；uC/OS-III 的任务在调用 OSTimeDlyHMSM 后会主动让出 CPU，调度器去运行其他任务，所以两个 LED 和串口才能看起来同时工作。移植过程中最容易出错的地方不是 API，而是底层：SysTick 被谁使用、PendSV 优先级、HAL_Delay 与 OS 节拍冲突、栈空间是否足够。这些问题逼着我去看启动文件、异常向量和端口汇编，而不是只会调用函数。

AHT20 的实验则让我体会到“会写 I2C 时序”和“会看数据手册”是两件事。软件 I2C 的起始、停止、ACK 并不难，但状态字节、忙碌位、CRC8、20 位数据的拼接和温湿度换算，任何一步出错都只能得到错误数据。把任务二改成传感器任务后，我进一步理解了 RTOS 的价值：传感器读取

需要等待 80 ms，如果裸机里用延时，整个系统都会被拖住；用 OSTimeDlyHMSM 让任务睡眠，LED 任务仍然能准时运行。

以后如果继续完善，我会把 AHT20 读取改成“触发—延时—读取”的状态机，避免在驱动里直接阻塞；串口输出可以加互斥量，防止多个任务同时打印；还可以增加看门狗、栈使用统计和 CPU 利用率统计，让系统更可靠。